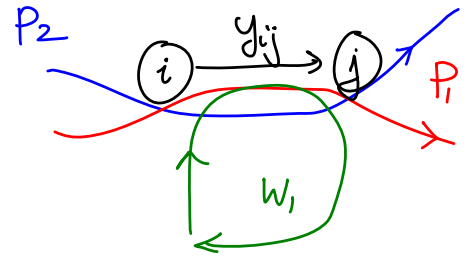


MATH 566: Lecture 10 (09/24/2026)

Today: * flow decomposition
* shortest path problem - assumptions, applications, algo

Recall: Flow decomposition...

Note that (i, j) can be part of multiple paths and/or cycles, but it is always a forward arc in each path and cycle.

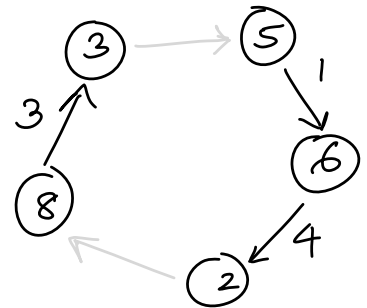
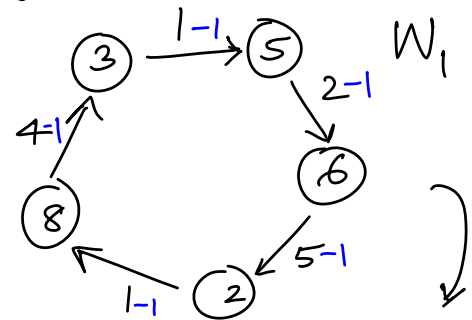


For a cycle W , we set $\Delta(W) = \min \{ y_{ij} \mid (i, j) \in W \}$.
↳ capacity of cycle W

Here, $\Delta(W_1) = 1$.

We set $f(W_1) = \Delta(W_1) = 1$.

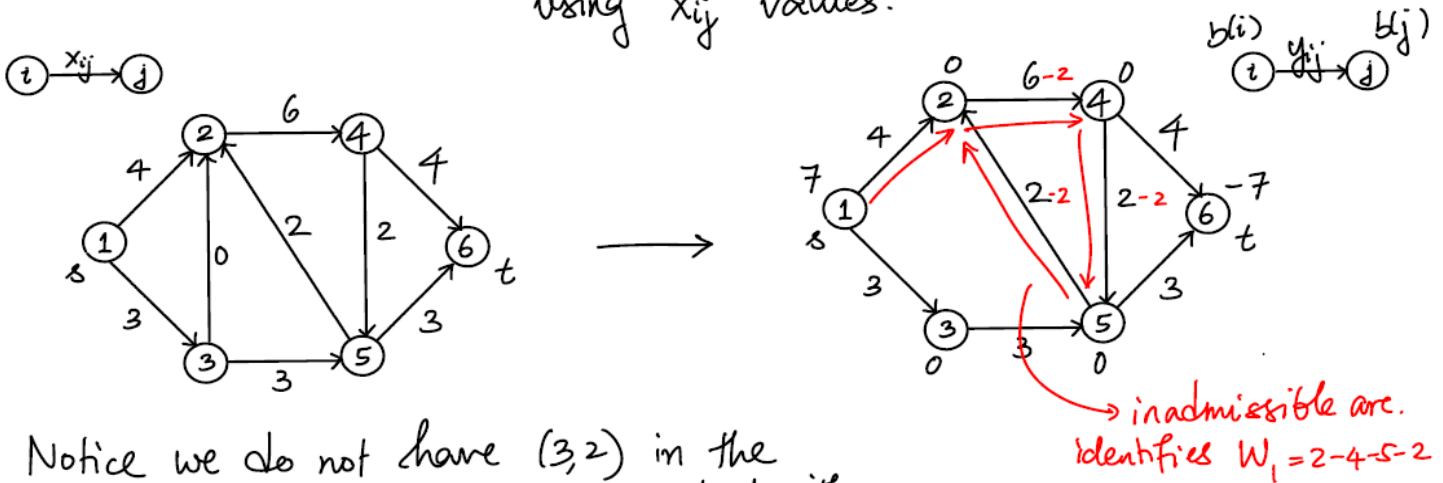
After update, $(3, 5)$ and $(2, 8)$ are removed from further consideration.



We run DFS repeatedly. If we find a directed cycle, we push its capacity of flow. Else if we find a path from a supply to a demand node, we push its capacity. We update the intermediate flows and the network, and continue. See the algorithm in the handout posted on the course web page.

Illustration

Consider the input flow. We start by initializing $b(i)$ and y_{ij} values using x_{ij} values.



Notice we do not have (3,2) in the starting network, as $y_{32} = 0$ to start with.

We maintain sets of supply and demand nodes. $\mathcal{S}, \mathcal{D}$.

$\mathcal{S} = [1], \mathcal{D} = [6]$ at the start.

Start with $s = 1$ (taken from $\mathcal{S}$).

We perform DFS. We identify cycle $W_1 = 2-4-5-2$. We set

$\mathcal{W} = \{W_1\}$. $\Delta(W_1) = \min\{6, 2, 2\} = 2$. Set $f(W_1) = 2$.

The updated network is shown here.

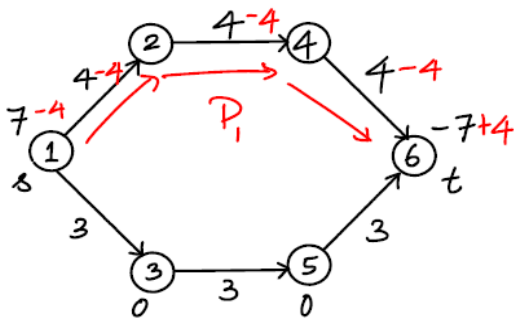
We still have $\mathcal{S} = [1], \mathcal{D} = [6]$.

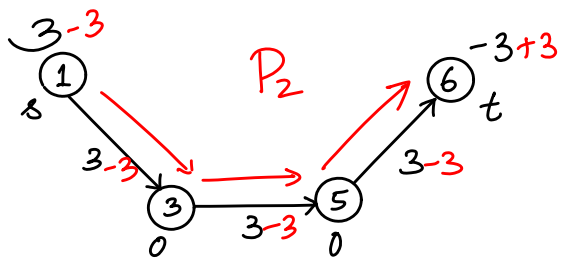
Starting with $s = 1$, we do DFS.

We identify $P_1 = 1-2-4-6$, with

$\Delta(P_1) = \min\{7, 4, 4, 4, 7\} = 4$.

We set $\mathcal{P} = \{P_1\}$, $f(P_1) = 4$.





$$S = [1], D = [6]$$

Start with $s=1$, do DFS. We find $P_2 = 1-3-5-6$ with

$$\Delta(P_2) = \min\{3, 3, 3, 3, \infty\} = 3.$$

We set $\mathcal{P} = \mathcal{P} \cup \{P_2\} = \{P_1, P_2\}$, and $f(P_2) = 3$.

After this iteration, all $b(i) = 0$, $y_{ij} = 0$. Thus we have decomposed the arc flow into flows along paths P_1, P_2 , and cycle W_1 .

We're trying to account for all y_{ij} and $b(i)$ for supply/demand nodes, by assigning amounts out of y_{ij} and $b(i)$ into $f(P)$ and $f(W)$. Eventually, we want to get $b(i) = 0 \forall i \in N$ and $y_{ij} = 0 \forall (i,j) \in A$, at which point, $f(P)$ and $f(W)$ for $P \in \mathcal{P}$ and $W \in \mathcal{W}$ account for all flows.

The input flow $\bar{x} = [x_{ij}]$ (vector of x_{ij} values) is assumed to satisfy flow balance, and bounds. But it need not necessarily be an optimal flow — notice that we do not worry about costs in this decomposition.

Similarly, the bounds (u_{ij}) do not play a direct part in the flow decomposition. Naturally, $f(P)$ and $f(W)$ cannot exceed y_{ij} , and hence x_{ij} values.

We finish the discussion on flow decomposition with a theorem:

Flow Decomposition Theorem Any feasible flow $\bar{x}$

$(x_{ij} \neq 0, (i,j) \in A)$ can be decomposed into

(a) the sum of flows in directed paths from supply to demand nodes, and

(b) the sum of flows around directed cycles.

The decomposition will have at most $m+n$ directed paths and cycles, out of which at most m are cycles.

IDEA Each step of the flow decomposition algorithm identifies a path P or a cycle W . Pushing $\Delta(P)$ or $\Delta(W)$ (capacity of P or W) will necessarily zero out at least one b_i or one y_{ij} .

Corollary Any circulation $\bar{x}$, i.e., flow $\bar{x}$ with $b_i = 0 \forall i$, can be decomposed into the sum of flows around at most n directed cycles.

Notice that we could use the bounds of $m+n$ on the number of cycles and paths for doing the complexity analysis of the flow decomposition algorithm.

We will use the flow decomposition theorem in the proofs of certain results on the shortest path problem, and also later on for other results.

Shortest Path (SP) Problem

Recall: length of a path P is sum of lengths of arcs in P .

We study the following generalization of the SP problem: → as previously introduced

Given: $G=(N,A)$ with costs c_{ij} for $(i,j) \in A$, and a source node s .

Goal: find shortest length directed paths from s to all $j \in N \setminus \{s\}$.

Assumptions

1. c_{ij} are integers.
For ease of analysis of complexity of algorithms.

2. There is a directed path from s to each $i \in N \setminus \{s\}$.

If there is no such path to a particular node j , we add an extra arc (s,j) with $c_{sj} = \infty$. If such an arc is part of the SP tree, we know that node is in fact not reachable from s via a directed path.

3. There is no negative cost cycle in G .

If there is such a cycle, going around it infinitely many times will decrease the total cost without limits! We will have to then impose extra restrictions that we use each arc at most once. But such restrictions make the problem hard to solve. Under this assumption, each arc would be used at most once in the SP tree.

In general, we do permit some C_{ij} 's to be negative, as long as there are no negative cost cycles.

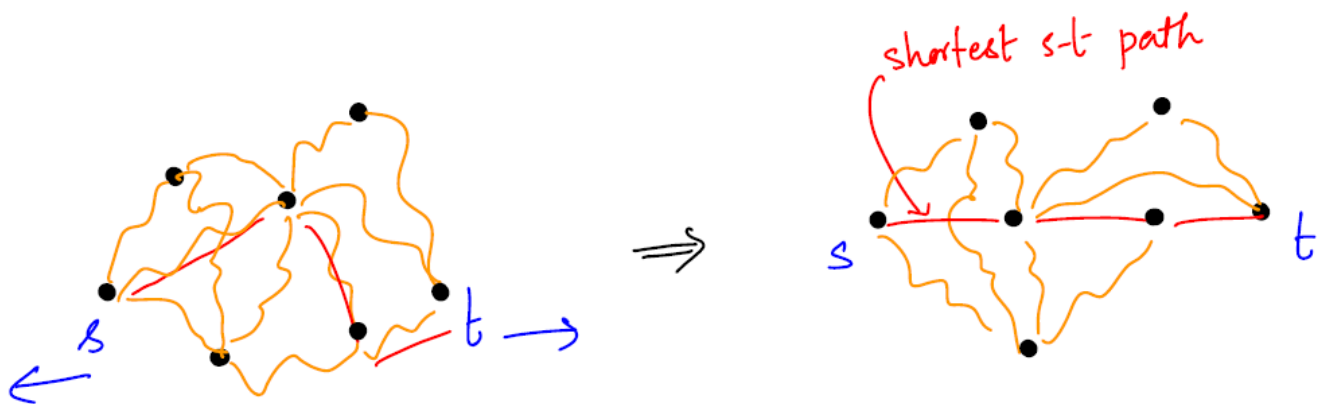
We first consider the case where all $C_{ij} \geq 0$. This case will be easier than the general case that allows $C_{ij} < 0$. As it turns out, we would use similar ideas to solve both cases.

An Analogy: The string model

Assume $C_{ij} \geq 0$.

Make a model of G with beads and strings. Each node gets a bead, and each arc gets an inelastic string of length C_{ij} .

Consider the s - t version of the shortest path problem. Imagine pulling apart the beads corresponding to the nodes s and t .



The set of strings that become taut (or tight) first represents an s - t shortest path.

Applications of the Shortest Path Problem

Recall: TeX paragraph spacing problem (AMO Problem 1.7).
Section 4.3 in AMO describes many more applications modeled as SP problems. We consider one case here.

The Knapsack Problem

A hiker wants to decide which items to include in her knapsack. She has to choose from n items. The knapsack can carry up to W lbs. Item i has weight w_i and utility u_i . The goal is to maximize total utility while not exceeding total weight W . We will model this problem as a shortest path problem.

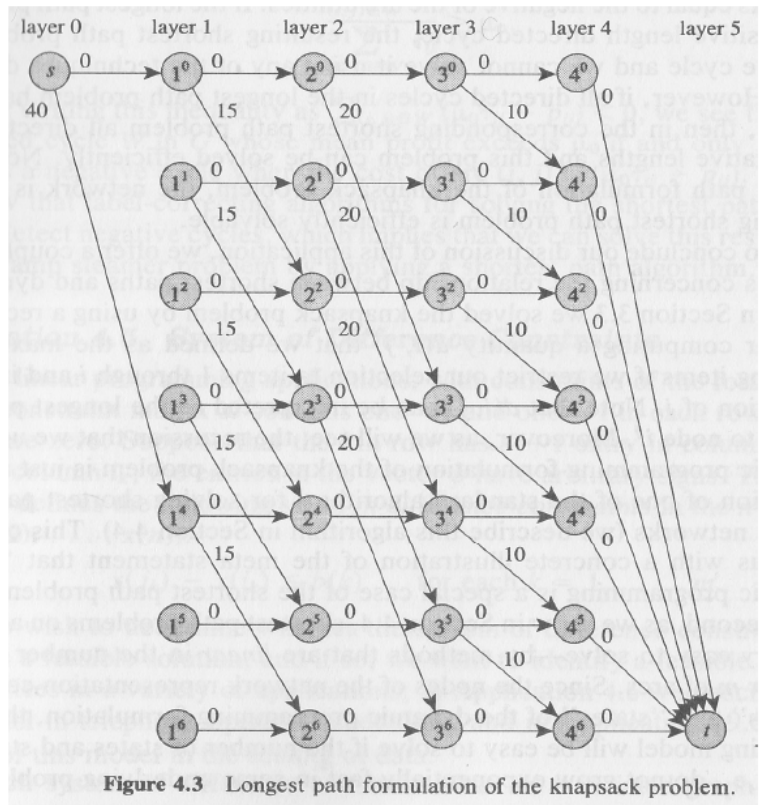
Example $n=4, W=6$

i	1	2	3	4
w_i	4	2	3	1
u_i	40	15	20	10

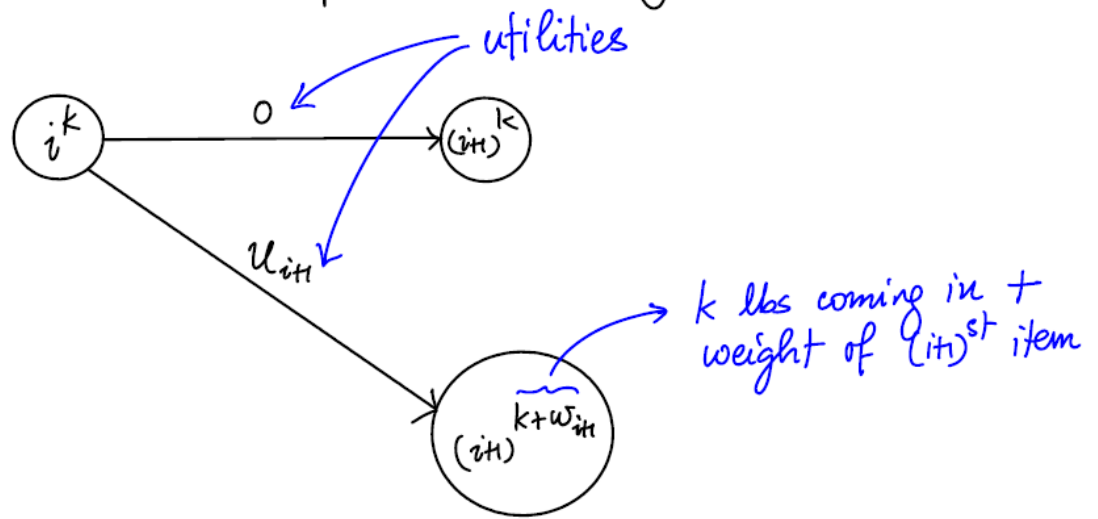
Notation

$(i)^k$: items $1, 2, \dots, i$ use k lbs (of total weight)

The figure illustrates a longest path formulation of this problem.



From node i^k , we have two outarcs, corresponding to the hiker including item i in the knapsack, or leaving it out.



From s , we add $(s, 0)$ with utility 0, and $(s, 1^{w_1})$ with utility u_1 . Finally, we connect all nodes from Layer n (last layer), i.e., nodes n^{w_i} for $w=1, 2, \dots, W$, to t with zero utility arcs.

Find the s - t longest path. When $u_i \geq 0 \forall i$, we can negate all utilities and solve the s - t SP problem.

The knapsack problem has many applications, including in cryptography, multiprocessor scheduling, fair division, etc. The model illustrated here (with layers corresponding to each item and levels corresponding to the total weight) is a typical instance of dynamic programming. Hence, under mild assumptions, many dynamic programming problems could be cast as shortest path problems.

Algorithms for Shortest Path Problem (Recall, we assume $c_{ij} \geq 0 \forall (i,j)$ for now)

Let $d(\cdot)$ denote a vector of temporary distance labels for the nodes. Thus, $d(i)$ = length of **some** path from s to i . Hence, $d(i)$ is an upper bound on the SP length from s to i .

A key step in all SP algorithms

Procedure UPDATE(i)

for each $(i,j) \in A(i)$ do

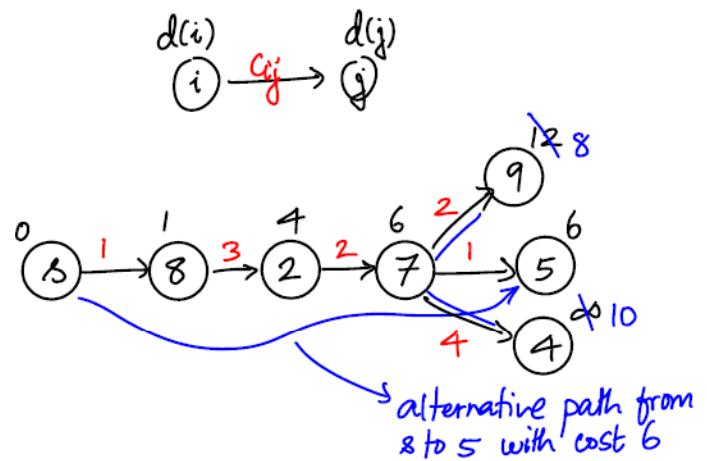
if $d(j) > d(i) + c_{ij}$ then

$d(j) := d(i) + c_{ij}$;

$\text{pred}(j) := i$;

end if

end for



UPDATE(7): We update $d(4)$ and $d(9)$, but do not change $d(5)$.

Note the similarity to the process of looking for admissibility of arcs from node i , and marking the heads of admissible arcs.

When the $d(i)$'s become permanent, they represent SP distances from s to i .

We now present the first algorithm for SP problems where $c_{ij} \geq 0$ for all (i,j) .

Dijkstra's algorithm (pronounced 'Daikstraa')

We maintain and update two (sub)sets of nodes.

S : set of "permanent" nodes ($d(i)$ is permanent)

$\bar{S} = N \setminus S$: set of nodes with temporary $d(i)$'s.

```

algorithm Dijkstra;
begin
   $S := \emptyset$ ;  $\bar{S} := N$ ;
   $d(i) := \infty$  for each node  $i \in N$ ;
   $d(s) := 0$  and  $\text{pred}(s) := 0$ ;
  while  $|S| < n$  do  $\rightarrow$  run till every  $d(i)$  is made permanent
  begin
    let  $i \in \bar{S}$  be a node for which  $d(i) = \min\{d(j) : j \in \bar{S}\}$ ;
     $S := S \cup \{i\}$ ;
     $\bar{S} := \bar{S} - \{i\}$ ;
    for each  $(i, j) \in A(i)$  do
      if  $d(j) > d(i) + c_{ij}$  then  $d(j) := d(i) + c_{ij}$  and  $\text{pred}(j) := i$ ;
    end;
  end;

```

Handwritten annotations:

- $\left. \begin{array}{l} d(i) := \infty \text{ for each node } i \in N; \\ d(s) := 0 \text{ and } \text{pred}(s) := 0; \end{array} \right\}$ initialization
- $\rightarrow$ run till every $d(i)$ is made permanent
- $\left. \begin{array}{l} \text{let } i \in \bar{S} \text{ be a node for which } d(i) = \min\{d(j) : j \in \bar{S}\}; \\ S := S \cup \{i\}; \\ \bar{S} := \bar{S} - \{i\}; \end{array} \right\}$ node selection
- $\left. \begin{array}{l} \text{for each } (i, j) \in A(i) \text{ do} \\ \text{if } d(j) > d(i) + c_{ij} \text{ then } d(j) := d(i) + c_{ij} \text{ and } \text{pred}(j) := i; \end{array} \right\}$ UPDATE(i)
- make i "permanent"* (pointing to the selection step)

Figure 4.6 Dijkstra's algorithm.

In words, pick node i with smallest $d(i)$ from $\bar{S}$, make it permanent, i.e., move it to S . Then run $\text{UPDATE}(i)$ by exploring the outarcs of node i .